

実装：NumPyだけでニューラルネットワーク

初回の図を自分のコードで再現する

rkskmt

1

部品はすべて揃っている

- アフィン変換・ReLU・シグモイド・交差エントロピー（最小単位／変換を重ねるの回）
- 勾配降下法（最適化の回）
- 全パラメータの勾配を出す backprop（前回）

📌 今日のゴール

講義の初回に見せた「機械が自力で見つけた曲がる境界」を、 **ライブラリなし（NumPyのみ）の自分のコード**で再現する。 **今日、新しい概念は1つも出てこない。** 組み立てるだけ。

2

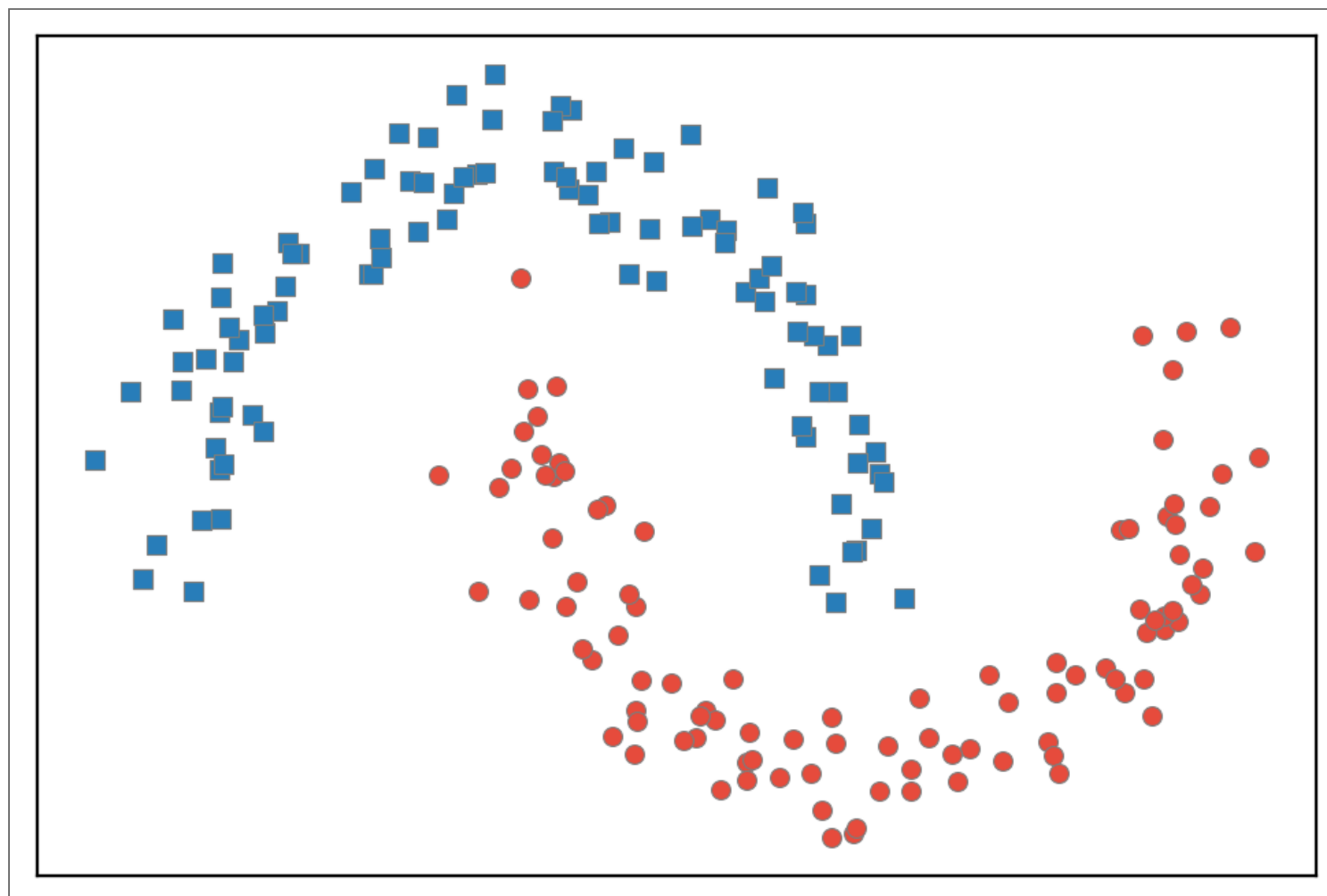
データ：あの三日月

Code

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4 from sklearn.datasets import make_moons
5 plt.rcParams["font.size"] = 11
6 plt.rcParams["figure.dpi"] = 100
7
8 X, y = make_moons(n_samples=200, noise=0.1, random_state=42)
9 print("X の形:", X.shape, " y の形:", y.shape)
10
11 plt.figure(figsize=(5, 3.4))
12 for cls, col, m in [(0, "#2980b9", "s"), (1, "#e74c3c", "o")]:
13     mask = y == cls
14     plt.scatter(X[mask, 0], X[mask, 1], c=col, marker=m, s=25, edgecolors="gray", lw=0.4)
15 plt.xticks([]); plt.yticks([])
16 plt.tight_layout()
17 plt.show()
```

Result

X の形: (200, 2) y の形: (200,)



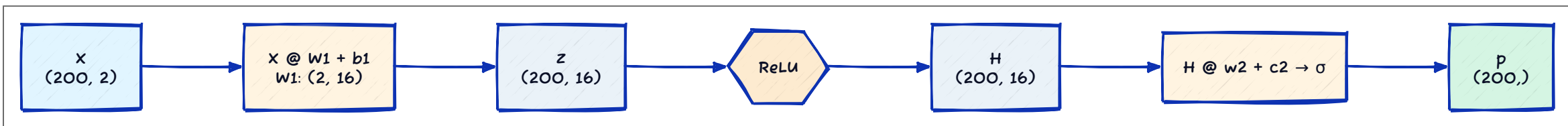
- 直線では分けられない、おなじみのデータ
- データの形は **(200, 2)** — 200点、2特徴量

設計：ループを行列に畳む

- 中間の変換を16個並べる。1個ずつ書くと16本の式 — **行列にまとめる**

記号	形	意味
W_1	$(2, 16)$	中間16変換ぶんの重み（列が1変換）
b_1	$(16,)$	中間のバイアス
w_2	$(16,)$	出力層の重み（多数決の票）
c_2	スカラー	出力層のバイアス

- 200点まとめて流すのも行列のまま： $Z = XW_1 + b_1$ は $(200, 16)$
- 「変換16個 × データ200点」が**たった1回の行列積**



4

Forward：4行

```
Code
1 def forward(X, W1, b1, w2, c2):
2     Z = X @ W1 + b1          # アフィン変換（全変換まとめて）
3     H = np.maximum(0, Z)     # ReLU（折る）
4     u = H @ w2 + c2          # 出力層（多数決）
5     p = 1 / (1 + np.exp(-u)) # シグモイド（確率に）
6     return Z, H, u, p
```

- 最小単位の回の forward と見比べると、**スカラーが行列になっただけ**
- Z, H も返すのは、**復路（backprop）で使うから**（前回の伏線）

5

Backward：前回の漸化式をそのまま書く

```
Code
1 def backward(X, y, Z, H, w2, p):
2     N = len(X)
3     delta_out = (p - y) / N          # 出力層の誤差 (p - y)
4     g_w2 = H.T @ delta_out           # ∂L/∂w2 = δ_out × 入力(H)
5     g_c2 = delta_out.sum()           # ∂L/∂c2 = δ_out
6     D_mid = np.outer(delta_out, w2) * (Z > 0) # δ_mid = δ_out · w2 · ReLU'(Z)
7     g_W1 = X.T @ D_mid               # ∂L/∂W1 = δ_mid × 入力(X)
8     g_b1 = D_mid.sum(axis=0)          # ∂L/∂b1 = δ_mid
9     return g_W1, g_b1, g_w2, g_c2
```

- 1行ずつ、前回の手計算と**完全に対応**している
 - **(p - y)**：シグモイド＋交差エントロピーの打ち消し
 - *** (Z > 0)**：ReLU' は 1 か 0
 - 「 δ × その層への入力」が各勾配

6

学習ループ：あとは勾配降下法

Code

```
1 rng = np.random.default_rng(0)
2 hidden = 16
3 W1 = rng.normal(0, 1.0, (2, hidden)) # ランダム初期化 (対称性を破る)
4 b1 = np.zeros(hidden)
5 w2 = rng.normal(0, 1.0, hidden)
6 c2 = 0.0
7 lr = 0.5
8
9 for i in range(1, 4001):
10     Z, H, u, p = forward(X, W1, b1, w2, c2)
11     g_W1, g_b1, g_w2, g_c2 = backward(X, y, Z, H, w2, p)
12     W1 -= lr * g_W1
13     b1 -= lr * g_b1
14     w2 -= lr * g_w2
15     c2 -= lr * g_c2
16
17     if i in (1, 100, 500, 2000, 4000):
18         eps = 1e-10
19         loss = -np.mean(y * np.log(np.clip(p, eps, 1)) +
20                        (1 - y) * np.log(np.clip(1 - p, eps, 1)))
21         acc = np.mean((p >= 0.5) == y)
22         print(f"更新 {i:5d}回 | Loss: {loss:.4f} | 正答率: {acc:.1%}")
```

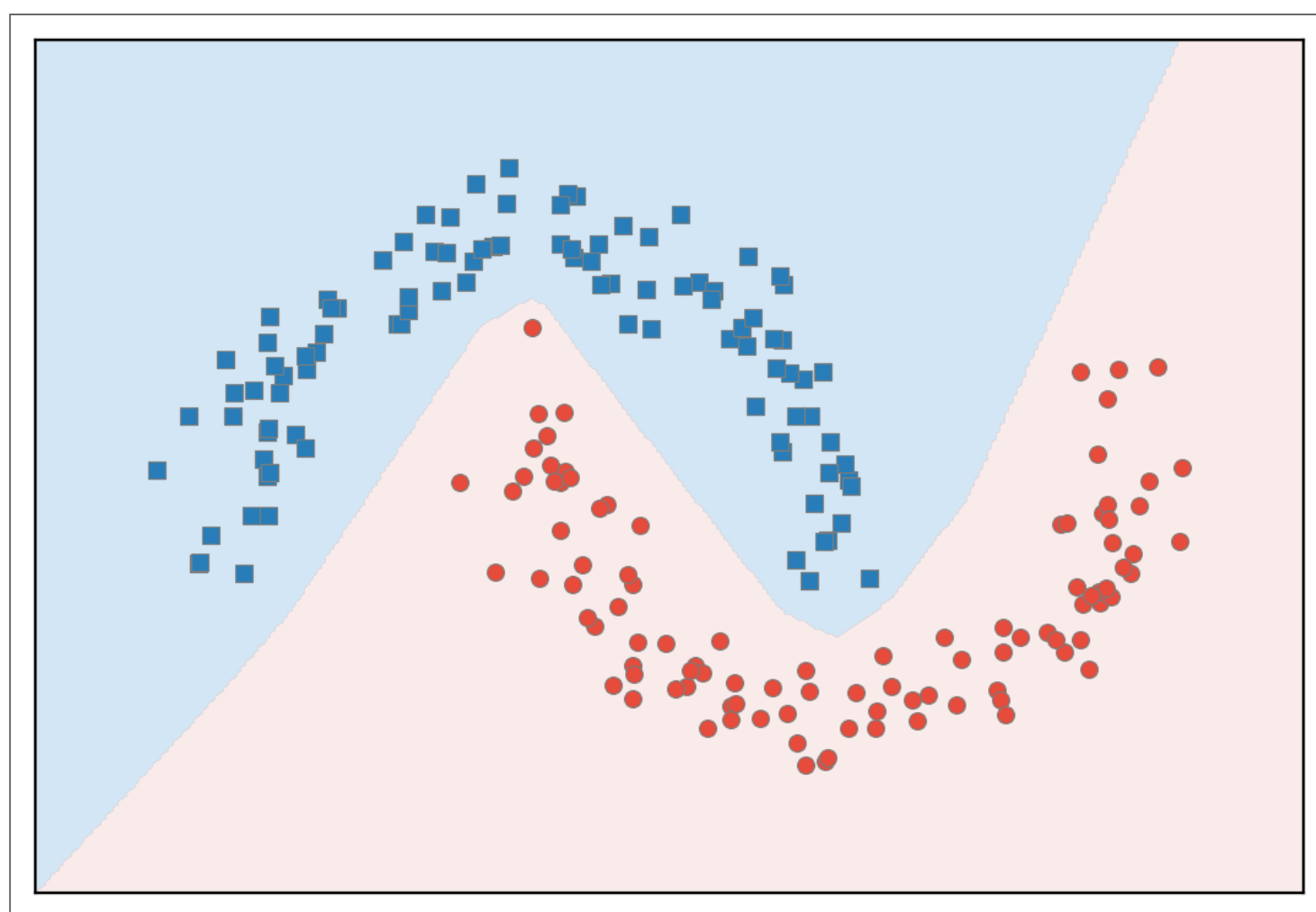
Result

更新	1回		Loss: 1.0894		正答率: 57.5%
更新	100回		Loss: 0.1741		正答率: 93.5%
更新	500回		Loss: 0.0315		正答率: 99.5%
更新	2000回		Loss: 0.0058		正答率: 100.0%
更新	4000回		Loss: 0.0023		正答率: 100.0%

- 正答率 **100%**。直線では87%が限界だったデータが、完全に分かれた

コールドオープン回収：初回の図、再現完了

▶ 描画コード

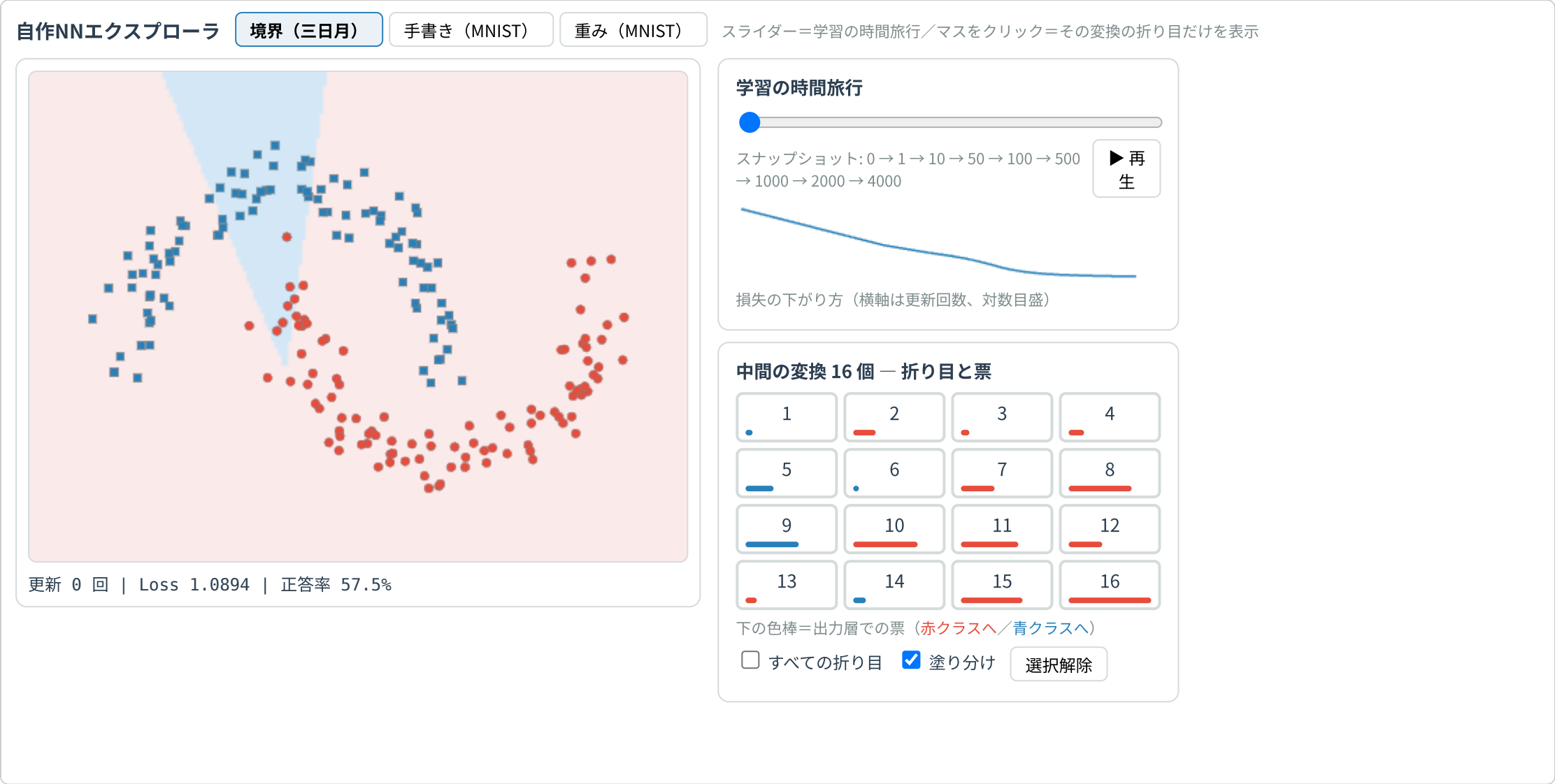


① 約束の回収

講義の初回、「数式も規則も誰も教えていないのに機械が見つけた境界」として見せた図。いまこの図は、**1行残らず自分が書いたコード**から出ている。中身に未知の部品はもうない。

学習の一部始終を再生する

いま回した学習ループ（**rng(0)**・lr 0.5・4000回）の途中経過を保存してある。▶ **再生**を押す：



エクスプローラを別タブで開く

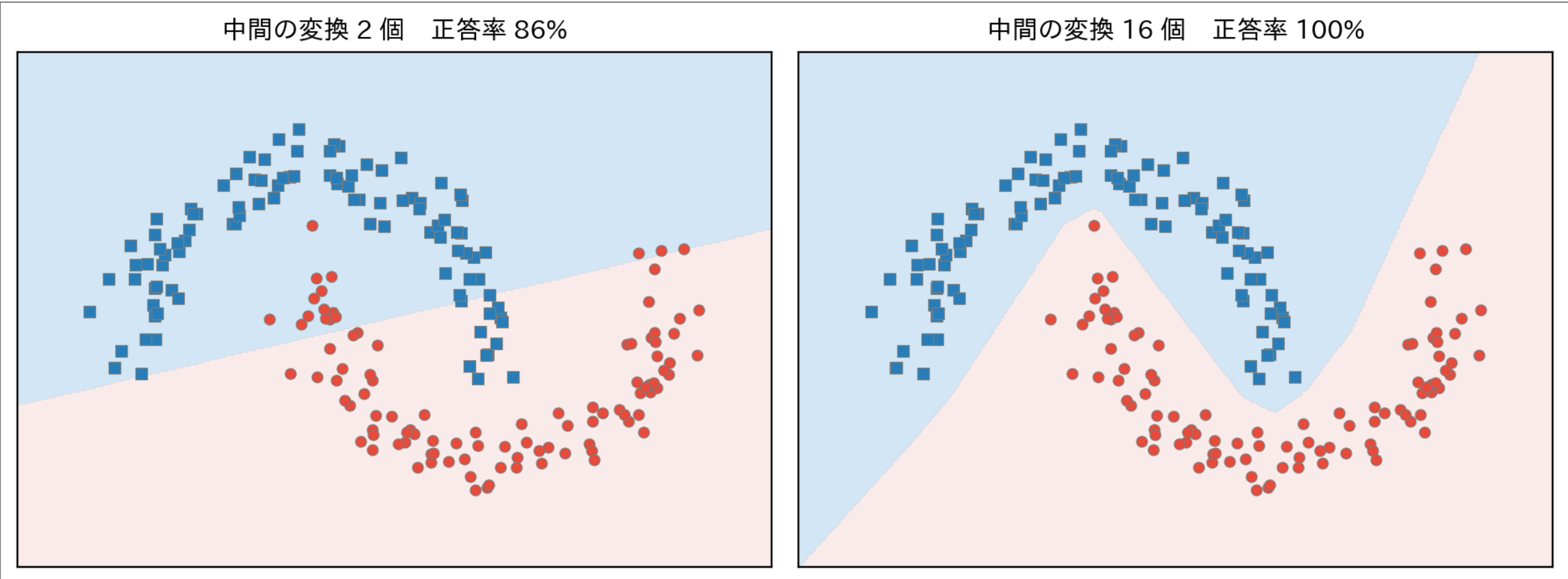
- 更新0回（ランダム初期値）の境界は**でたらめ**（正答率 57.5%）。それが 1000 回で 100% に届く
- 変換のマスをクリックしたままスライダーを動かす — **折り目が引越していく** のが見える
- 更新 500 回（99.5%）→ 1000 回（100%）：最後の1匹を拾うための微調整

9

折り目の数で遊ぶ

❗ クイズ

中間の変換16個で 100% だった。**2個**まで減らしたら、正答率はどこまで落ちると思う？



- 中間2個では折り目が足りない。16個なら滑らかに曲がる — **「変換を重ねる」の回の結論** を、自作コードで追試できた

10



- 学習ループのコード を書き換えて実行（境界の描画コードまで続けて実行すると形の変化が見える）
- 壊れても全部自分のコードなので、原因も自分で追える

1 折り目を2本に減らす

`hidden = 2`

2 盛大に増やす

`hidden = 100`

3 乱数を変えて別の谷へ

`default_rng(0) → default_rng(1)`

💡 ここで観察

- 乱数シードを変えると、正答率は同じでも **境界の形が微妙に違う** — 初期値しだいで別の谷に落ちている。 ローカルミニマムの罠（参考資料） が、本物のNNで起きている瞬間

11

まとめ

- NumPy の行列演算で、**forward 4行・backward 6行**の多層ネットワークができた
- 中身は既習の部品のみ：アフィン変換・ReLU・ $(p - y) \cdot \delta$ の漸化式・勾配降下法
- 初回のコールドオープンの図を、自分のコードで再現した

📄 次回予告

全部自分で書けるようになった**からこそ**、次回はライブラリ（PyTorch）で楽をする。最大の見せ場：手計算した backprop の勾配を、**`loss.backward()`** が一発で言い当てる。

12



自作した **forward**・**backward**・学習ループを指しながら、入力から曲がった境界が育つまでを60秒で説明せよ。

0. 線形ではない変換が入る箇所
1. 誤差が出力から第1層へ届く箇所
2. 学習で実際に書き換わる数字

13

練習問題

以下の問題を **Pythonコード** で解け。

問題1：自作 **backward** を数値微分で検品する

自分の書いた **backward** が正しい保証はまだない。前回の検算法（数値微分）を、行列版にも適用せよ。

- ランダム初期化した直後のパラメータで、**backward** が返す勾配 **g_W1[0, 0]** を表示せよ
- W1[0, 0]** だけを $\epsilon = 10^{-6}$ ずらして損失を再計算し、数値微分
$$\frac{L(W_1[0,0] + \epsilon) - L(W_1[0,0])}{\epsilon}$$
 を求めよ
- 2つが一致することを確認せよ

ヒント：損失（交差エントロピー）の計算は学習ループの中にある。関数に切り出すと楽。

14

問題1の解答例

▶ 解答例を見る

```
Result
|
|   backprop:  ∂L/∂W1[0,0] = -0.003024
|   数値微分:  ∂L/∂W1[0,0] = -0.003024
|
```

- 一致すれば、少なくともこの成分の backprop は正しい
- 実務のライブラリ開発でも、autograd の実装はこうやって **数値微分と突き合わせてテスト** されている

15